

LAPORAN UJIAN TENGAH SEMESTER
SISTEM PENGADUAN MAHASISWA

Dosen Pengampu: Muhammad Panji Muslim, S.Pd., M.Kom.



MATA KULIAH:
PEMBANGUNAN PERANGKAT LUNAK ORIENTASI SERVICE (SE.2)

Disusun Oleh:
Zaskia Alifa Mardatilla (2410511156)

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL VETERAN JAKARTA

2026

A. Pendahuluan

Sistem pengaduan mahasiswa konvensional masih banyak dilakukan secara manual, baik melalui surat fisik maupun tatap muka langsung dengan pihak terkait. Proses ini tidak efisien karena sulit dilacak, lambat dalam penanganan, dan tidak transparan bagi mahasiswa yang mengadu. Oleh karena itu, dibangunlah sistem pengaduan mahasiswa berbasis *service-oriented architecture* yang memungkinkan pengaduan dilakukan secara digital dengan alur yang terstruktur dan dapat dipantau secara *real-time*.

B. Arsitektur Sistem

Sistem dibangun menggunakan arsitektur *microservice* yang terdiri dari 3 *service* independen dan 1 API Gateway sebagai *single entry point*. Setiap *service* memiliki database yang terpisah dan berkomunikasi melalui REST API.

Path	Service Tujuan	Port	Teknologi
/auth/	Auth-Service	3001	Node.js
/pengaduan/	Pengaduan-Service	3002	PHP CodeIgniter 4
/disposisi/	Disposisi-Service	3003	Node.js

Seluruh request dari klien hanya boleh masuk melalui API Gateway di port 3000. Gateway melakukan validasi JWT dan *rate limiting* sebelum meneruskan *request* ke *service* tujuan.

C. Justifikasi Pemisahan Service

Pada pendekatan monolitik, seluruh fitur autentikasi, pengaduan, dan disposisi berada dalam satu aplikasi dengan satu database. Pendekatan ini memiliki beberapa kelemahan:

1. Jika satu modul mengalami error, seluruh sistem ikut terganggu
2. Scaling harus dilakukan untuk seluruh aplikasi meskipun hanya satu modul yang membutuhkan *resource* lebih
3. Perubahan kecil pada satu fitur mengharuskan *deployment* ulang seluruh aplikasi
4. Tim dengan keahlian berbeda tidak bisa bekerja secara paralel dan independen

Dengan pendekatan *microservice* yang diterapkan pada sistem ini, setiap *service* dapat dikembangkan, dideploy, dan di-scale secara independen tanpa mempengaruhi *service* lain.

Justifikasi *auth-service* dipisah karena autentikasi adalah *cross-cutting concern* yang bisa digunakan oleh banyak *service* lain. Dengan memisahkan *auth service*, logika JWT dan OAuth tidak tercampur dengan logika bisnis pengaduan. Selain itu, jika di masa depan ada

service baru yang ditambahkan, service tersebut cukup berkomunikasi dengan *auth service* tanpa perlu mengimplementasikan ulang logika autentikasi.

Justifikasi pengaduan service dipisah disebabkan pengaduan merupakan domain bisnis utama sistem ini dengan logika dan data yang kompleks. Memisahnya memungkinkan tim backend PHP untuk mengembangkan service ini secara independen menggunakan framework CodeIgniter 4 tanpa harus menyesuaikan dengan teknologi service lain. Database pengaduan juga terpisah sehingga perubahan skema tidak mempengaruhi service lain.

Justifikasi disposisi service dipisah karena alur disposisi memiliki logika tersendiri yaitu meneruskan pengaduan ke unit terkait dan mengupdate statusnya. Dengan memisahkan disposisi service, alur bisnis ini dapat dikembangkan secara independen. Service ini juga mendemonstrasikan komunikasi inter-service dengan mengkonsumsi pengaduan service melalui HTTP request, sesuai dengan prinsip microservice.

D. Penjelasan Service

1. Auth-Service (Port 3001)

Auth service bertanggung jawab atas seluruh proses autentikasi dan otorisasi. Dibangun menggunakan Node.js dan Express dengan implementasi JWT dan Google OAuth 2.0.

Fitur yang diimplementasikan:

- a) Registrasi user baru dengan enkripsi password menggunakan bcryptjs
- b) Login menggunakan email dan password yang menghasilkan access token dan refresh token
- c) Access token memiliki masa berlaku 15 menit, refresh token 7 hari
- d) Endpoint refresh token untuk memperbarui access token yang expired
- e) Endpoint logout dengan mekanisme token blacklist sehingga token yang sudah logout tidak bisa digunakan kembali
- f) Login alternatif menggunakan Google OAuth 2.0 dengan Authorization Code Flow
- g) Data user dari Google (nama, email, foto profil) disimpan ke database lokal dengan flag `oauth_provider`

2. Pengaduan_Service (Port 3002)

Pengaduan service dibangun menggunakan PHP dengan framework CodeIgniter 4 mengikuti pola MVC. Pemisahan layer dilakukan secara jelas:

- a) Model (PengaduanModel.php): menangani query database dan validasi data
- b) Controller (PengaduanController.php): menangani logika bisnis dan response
- c) Routes (Routes.php): mendefinisikan endpoint dan mapping ke controller

Fitur yang diimplementasikan:

- a) CRUD pengaduan dengan kategori akademik dan non-akademik
- b) Validasi input pada layer controller menggunakan validation rules CodeIgniter
- c) Paging menggunakan parameter page dan per_page pada endpoint listing
- d) Filtering berdasarkan kategori dan status pengaduan
- e) Rating kepuasan (1-5) setelah pengaduan diselesaikan
- f) HTTP method yang semantik (GET, POST, PUT, DELETE) dengan response code yang sesuai

Database pengaduan_db memiliki 4 tabel ter-relasi: pengaduan, unit, disposisi, dan follow_up.

3. Disposisi-Service (Port 3003)

Disposisi service dibangun menggunakan Node.js dan Express. Service ini mengkonsumsi pengaduan service melalui HTTP request menggunakan library Axios, mendemonstrasikan komunikasi inter-service. Fitur yang diimplementasikan:

- a) Mengambil data pengaduan dari pengaduan service dan memfilter yang belum selesai
- a) Mengupdate status pengaduan menjadi diproses melalui pengaduan service
- b) Mengupdate status pengaduan menjadi selesai melalui pengaduan service

4. API Gateway (PORT 3000)

API Gateway berfungsi sebagai single entry point untuk seluruh request dari klien. Dibangun menggunakan Node.js dan Express. Fitur yang diimplementasikan:

- a) Routing request ke service tujuan berdasarkan path
- b) Validasi JWT di sisi gateway sebelum request diteruskan ke service pengaduan dan disposisi
- c) Rate limiting sebesar 60 request per menit per IP menggunakan express-rate-limit
- d) Error handling apabila service tujuan tidak tersedia

E. Database

Setiap service memiliki database yang terpisah sesuai prinsip microservice. Tidak ada tabel yang digunakan bersama antar service.

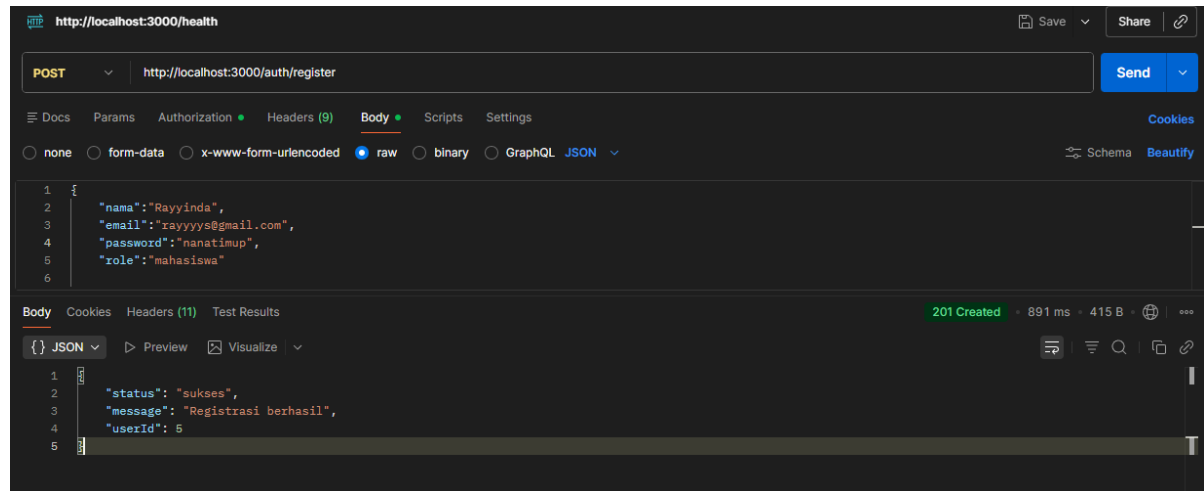
- a) auth_db: Tabel users menyimpan data pengguna termasuk informasi OAuth (oauth_provider, oauth_id, foto_profil) untuk mendukung login via Google.
- b) pengaduan: Terdiri dari 4 tabel ter-relasi:
 - pengaduan: menyimpan data pengaduan mahasiswa
 - unit: menyimpan data unit terkait yang menangani pengaduan
 - disposisi: menyimpan data disposisi pengaduan ke unit
 - follow_up: menyimpan riwayat tindak lanjut pengaduan

F. Implementasi Keamanan

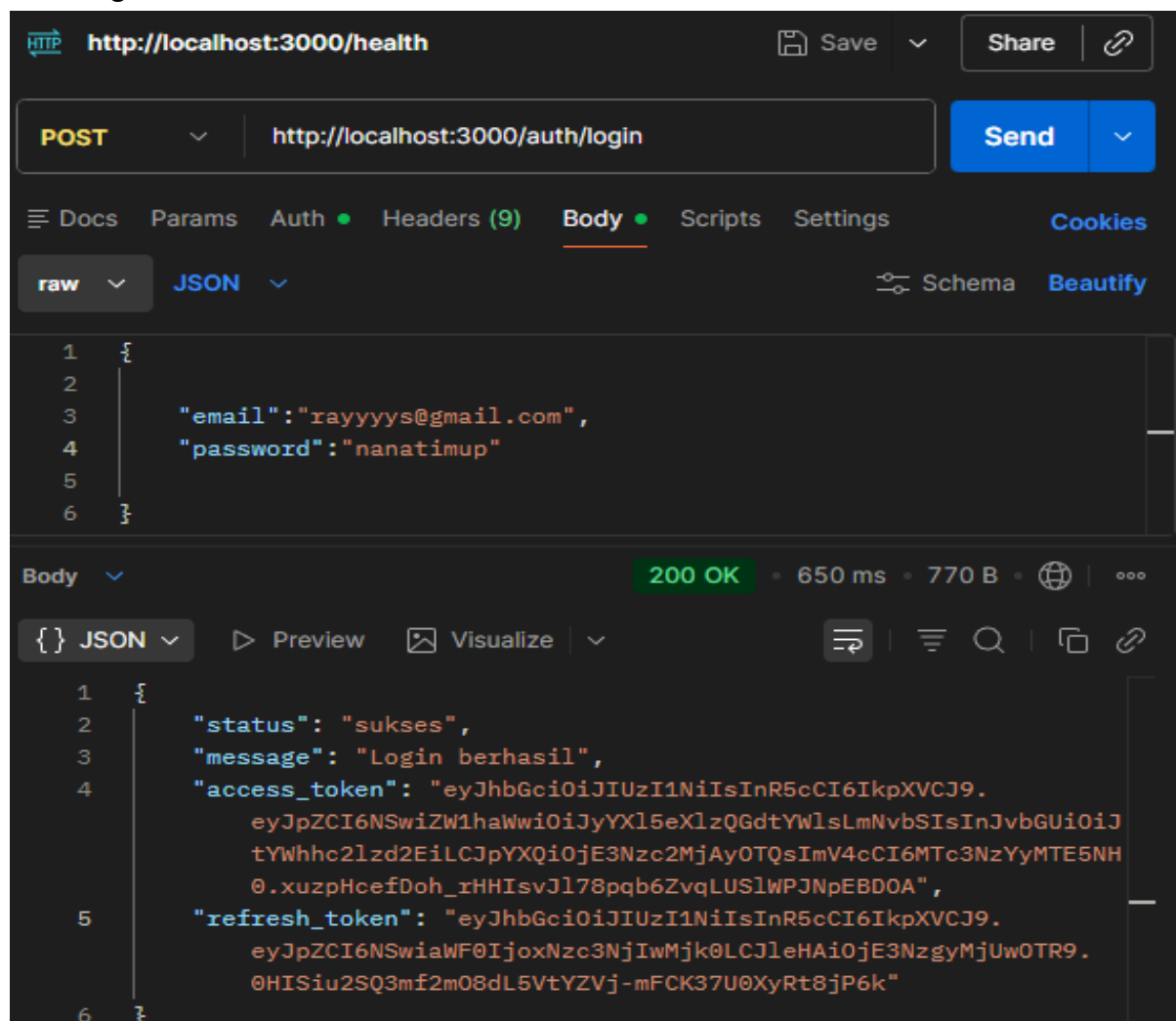
- a) JWT access token dengan masa berlaku 15 menit mencegah penyalahgunaan token yang bocor
- b) Refresh token 7 hari memungkinkan user tetap login tanpa harus input password berulang
- c) Token blacklist saat logout memastikan token yang sudah tidak digunakan tidak bisa dipakai kembali
- d) Google OAuth 2.0 menggunakan Authorization Code Flow (bukan Implicit Flow yang sudah deprecated)
- e) JWT validation dilakukan di gateway sebelum request diteruskan ke service, bukan per-endpoint secara manual
- f) Rate limiting 60 request per menit per IP mencegah abuse dan serangan brute force

1. Post : Registrasi akun

1. Post : Registrasi akun



2. Post: Login akun



3. Post: Refresh Token

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/auth/refresh`. The request body is a JSON object with a `refresh_token`. The response is a 200 OK status with a JSON body containing `status`, `access_token`, and `refresh_token`.

```
POST http://localhost:3000/auth/refresh
```

```
{
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NSwiaWF0IjoxNzc3NjIwMjk0LCJleHAiOjE3NzgyMjUwOTR9.0HISiu2SQ3mf2m08dL5VtYZVj-mFCK37U0XyRt8jP6k"
}
```

Body 200 OK • 140 ms • 523 B

```
{
  "status": "sukses",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NSwiaWF0IjoxNzc3NjIwMzgzLCJleHAiOjE3Nzc2MjEyODV9.ZaRj4N5o5vvWB54ajHQb5qYZ0xVEmBfts7YGLqxvPhk"
}
```

4. Google OAuth

The screenshot shows a web browser displaying a Google OAuth callback URL. The response is a JSON object containing user information and tokens.

```
localhost:3001/auth/google/callback?iss=https%3A%2F%2Faccounts.google.com&code=4%2F0AeoWuM-A_IgmXG3advA6lyc0xzDyzoxtaqhInb3018ruDEEOC-nFmMvx4CYt...
```

```
{
  "status": "sukses",
  "message": "Login Google berhasil",
  "user": {
    "nama": "zaskia alifa",
    "email": "zascialf2810@gmail.com",
    "foto_profil": "https://lh3.googleusercontent.com/a/ACg8ocJv0v_bw-AsIus2qh2IzDAmxvz0St1IIErhY2id9y09VKwM15Q=s96-c",
    "role": "mahasiswa",
    "oauth_provider": "google"
  },
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIwMjI1hahwI0IjYXNjaWVsZjI4MTBAZ21hahwY29tIiwicm9sZSI6Im1haGFzaXN3YSIsIm1hdCI6MTc3NzYyMTc0NywiZXhwIjoxNzc3NjIwMzgzLCJleHAiOjE3NzgyMjUwOTR9.eyJpZCI6MTIwMjI1hahwI0IjYXNjaWVsZjI4MTBAZ21hahwY29tIiwicm9sZSI6Im1haGFzaXN3YSIsIm1hdCI6MTc3NzYyMTc0NywiZXhwIjoxNzc3NjIwMzgzLCJleHAiOjE3NzgyMjUwOTR9.ZSKOIsCapPit6iR8dxrDFd9K6-PVkpDwSvPiGKO6E6Es"}
}
```

5. Post: Logout akun

HTTP **POST** `http://localhost:3000/auth/logout` Save Share

Docs Params Auth **Headers (9)** Body • Scripts Settings Cookies

Headers 8 hidden

	Key	Value	Bulk Edit	Presets
☒	Authorization	Bearer eyJhbGciOiJIUzI1...		
	Key	Value	Description	

Body 200 OK • 30 ms • 395 B • Preview Visualize

```
{
  "status": "sukses",
  "message": "Logout berhasil"
}
```

6. Get: Mengambil semua data pengaduan

GET `http://localhost:3000/pengaduan` Send

Docs Params Auth **Headers (9)** Body • Scripts Settings Cookies

Headers 8 hidden

	Key	Value	Bulk Edit	Presets
☒	Authorization	Bearer eyJhbGciOiJIUzI1...		
	Key	Value	Description	

Body 200 OK • 4.78 s • 410 B • Preview Visualize

```
{
  "status": "sukses",
  "total": 0,
  "page": 1,
  "per_page": 10,
  "data": []
}
```


7. Post: Membuat data pengaduan baru

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/pengaduan`. The request body is a JSON object with the following fields:

```
{
  "judul": "Nilai tidak keluar",
  "deskripsi": "Nilai semester ganjil belum muncul di SIAKAD"
}
```

The response is a 201 Created status with a response time of 1.28 s and a body size of 664 B. The response body is a JSON object:

```
{
  "status": "sukses",
  "message": "Pengaduan berhasil dibuat",
  "data": {
    "id": "1",
    "judul": "Nilai tidak keluar",
    "deskripsi": "Nilai semester ganjil belum muncul di SIAKAD UPNVJ",
    "kategori": "akademik",
    "status": "menunggu",
    "rating": null,
    "user_id": "6",
    "created_at": "2026-05-01 08:28:42",
    "updated_at": "2026-05-01 08:28:42"
  }
}
```

8. Get: Mengambil data pengaduan dengan ID

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/pengaduan/1`. The request headers include an Authorization header with a Bearer token. The response is a 200 OK status with a response time of 125 ms and a body size of 621 B. The response body is a JSON object:

```
{
  "status": "sukses",
  "data": {
    "id": "1",
    "judul": "Nilai tidak keluar",
    "deskripsi": "Nilai semester ganjil belum muncul di SIAKAD UPNVJ",
    "kategori": "akademik",
    "status": "menunggu",
    "rating": null,
    "user_id": "6",
    "created_at": "2026-05-01 08:28:42",
    "updated_at": "2026-05-01 08:28:42"
  }
}
```

9. Put: Update data pengaduan ke 'diproses' dengan ID

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/pengaduan/1`
- Method:** `PUT`
- Request Body (JSON):**

```
{
  "status": "diproses"
}
```
- Response Status:** `200 OK` (200 ms, 661 B)
- Response Body (JSON):**

```
{
  "status": "sukses",
  "message": "Pengaduan berhasil diupdate",
  "data": {
    "id": "1",
    "judul": "Nilai tidak keluar",
    "deskripsi": "Nilai semester ganjil belum muncul di SIAKAD UPNVJ",
    "kategori": "akademik",
    "status": "diproses",
    "rating": null,
    "user_id": "6",
    "created_at": "2026-05-01 08:28:42",
    "updated_at": "2026-05-01 08:37:21"
  }
}
```

10. Delete: Hapus data pengaduan

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:3000/pengaduan/1`
- Method:** `DELETE`
- Request Body (JSON):**

```
{
  "status": "diproses"
}
```
- Response Status:** `200 OK` (154 ms, 406 B)
- Response Body (JSON):**

```
{
  "status": "sukses",
  "message": "Pengaduan berhasil dihapus"
}
```

11. Put: Rating pengaduan

HTTP **PUT** `http://localhost:3000/pengaduan/2/rating` Save Share

PUT `http://localhost:3000/pengaduan/2/rating` Send

Docs Params Auth Headers (9) **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "rating": 5
3 }
```

Body 200 OK • 139 ms • 710 B • 🌐 | ⋮

{ } JSON Preview Visualize

```
1 {
2   "status": "sukses",
3   "message": "Rating berhasil ditambahkan",
4   "data": {
5     "id": "2",
6     "judul": "FIKLAB 302 tidak bisa diakses",
7     "deskripsi": "Sudah seminggu mahasiswa tidak bisa menggunakan FIKLAB 302 tanpa keterangan yang jelas",
8     "kategori": "non-akademik",
9     "status": "selesai",
10    "rating": "5",
11    "user_id": "5",
12    "created_at": "2026-05-01 08:40:46",
13    "updated_at": "2026-05-01 08:41:58"
14  }
15 }
```

12. Get: Mengambil data disposisi

GET `http://localhost:3000/disposisi` Send

Docs Params Auth **Headers (9)** Body Scripts Settings Cookies

Headers 8 hidden

	Key	Value	...	Bulk Edit	Presets
☒	Authorization	Bearer eyJhbGciOiJIUzI1...			

Body 200 OK • 761 ms • 911 B • 🌐 | ⋮

{ } JSON Preview Visualize

```
1 {
2   "status": "sukses",
3   "message": "Data pengaduan untuk disposisi",
4   "data": [
5     {
6       "id": "3",
7       "judul": "Nilai tidak keluar",
8       "deskripsi": "Nilai semester ganjil belum muncul di portal",
9       "kategori": "akademik",
10      "status": "menunggu",
11      "rating": null,
12      "user_id": "1",
13      "created_at": "2026-05-01 08:50:45",
14      "updated_at": "2026-05-01 08:50:45"
15    }
16  ]
17 }
```

13. Put: Memproses data disposisi dengan ID

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/disposisi/3/proses`. The request headers include an Authorization token. The response is a 200 OK status with a JSON body indicating success.

Key	Value
Authorization	Bearer eyJhbGciOiJIUzI1...

```
{
  "status": "sukses",
  "message": "Pengaduan berhasil didisposisi",
  "data": {
    "id": "3",
    "judul": "Nilai tidak keluar",
    "deskripsi": "Nilai semester ganjil belum muncul di portal",
    "kategori": "akademik",
    "status": "diproses",
    "rating": null,
    "user_id": "1",
    "created_at": "2026-05-01 08:50:45",
    "updated_at": "2026-05-01 08:55:19"
  }
}
```

14. Put: Menyelesaikan disposisi dengan ID

The screenshot shows a REST client interface with a PUT request to `http://localhost:3000/disposisi/3/selesai`. The request headers include an Authorization token. The response is a 200 OK status with a JSON body indicating success.

Key	Value
Authorization	Bearer eyJhbGciOiJIUzI1...

```
{
  "status": "sukses",
  "message": "Pengaduan berhasil diselesaikan",
  "data": {
    "id": "3",
    "judul": "Nilai tidak keluar",
    "deskripsi": "Nilai semester ganjil belum muncul di portal",
    "kategori": "akademik",
    "status": "selesai",
    "rating": null,
    "user_id": "1",
    "created_at": "2026-05-01 08:50:45",
    "updated_at": "2026-05-01 08:55:39"
  }
}
```